COMP-8117 · SPLASH PROJECT PLAN · OPTION B

"Paid, But Never Shipped"

Making sure an online store never loses or double-processes a paid order

Team: *[names]* **Duration:** 10 weeks **Defence:** December 2026 **Version:** 1.0

01 Problem

When a customer pays in an online store, the payment provider (e.g. Stripe) tells the store's backend with a **webhook**, a message that says "order #123 is paid". If the backend is down, slow or mid-deploy at that moment, the message fails. The provider retries for a while, then gives up. Two things go wrong for small stores:

- **Lost orders:** the customer was charged, but the order is never fulfilled. That means an angry customer, a refund, sometimes a chargeback.
- **Duplicate orders:** a retry arrives after the first message did get through, and the order ships twice.

> **Problem statement.** How can a small online store guarantee that every paid order is processed **exactly once**, even when its backend crashes, restarts or is being deployed?

Impact. For the store: no lost revenue, no double shipping, no manual order-fixing. For customers: they get what they paid for. It's a real, common problem, and the fix is simple, standard engineering.

02 How we run the project: a simulated DevOps team

We act as the engineering team of a fictional small online store, **"ShopCo"**, with a Black Friday sale coming up. Our job is to make order processing reliable before the sale.

Constraint	Value
Deadline	"Black Friday" = our December defence
Team	3 part-time engineers, 10 weeks
Budget	≈ $110 total, hard cap $200/month (see §06)
Non-negotiables	0 lost orders · 0 duplicate orders · every change goes dev → staging → prod

03 Traffic projection

We simulate ShopCo's sale with Stripe **test mode** (free, no real money). Our load generator creates test payments at each phase's rate. During the peak we deliberately crash and redeploy the backend.

Phase	Expected orders	What we simulate	What must happen
Normal day	~200 / day	1 order / min	Every order processed within seconds
Sale peak (first hour)	~3,000 / hour	50 orders / min for 1 hour	Backlog forms, then drains; nothing lost
Chaos during peak	—	Kill the backend 3× and deploy once mid-peak	0 lost, 0 duplicates

Success criteria

Criterion	Target	Evidence
Lost orders	0	Stripe payments vs. orders in our DB
Duplicate orders	0	Unique order-ID check
Time from payment to order processed	p95 < 30 s normally; backlog clears < 10 min after a crash	Dashboard
Before vs. after	Same chaos test on a naive backend loses / duplicates orders	Side-by-side results

04 Architecture

Stripe (test mode)
Sends "payment succeeded" webhooks

→

Webhook inbox
Tiny endpoint: verify signature, save event, reply 200 instantly

→

Events table
Postgres. The event ID is unique, so duplicates are ignored

→

Order worker
Processes events, retries with backoff, sends failures to a dead-letter list

→

Orders
Created exactly once

Why this works: the inbox does almost nothing, so it almost never fails, and once an event is saved it can't be lost. The unique event ID makes duplicates harmless (**idempotency**). A **replay** command re-processes anything in the dead-letter list. **Baseline** for comparison: a naive backend that processes the order directly inside the webhook call.

05 Environments: dev, staging, prod

Dev

- **Runs:** local Docker Compose (API, worker, Postgres)
- **Webhooks:** Stripe CLI forwards test events to the laptop
- **Cost:** $0

Staging

- **Runs:** same cloud setup as prod
- **Purpose:** all load and chaos tests run here
- **Deploys:** on merge to `main`

Prod

- **Runs:** the "Black Friday" defence demo
- **Deploys:** manual promotion of a release that passed staging
- **Alerts:** dead-letter list not empty

Rollback: redeploy the previous tagged image in one command. Saved events are untouched by a rollback, so no orders are lost during it.

06 Resources and budget

Resource	Choice	Estimate
API + worker	2 small containers per env (Fargate ARM or App Runner), ~8 weeks	$45
Database	Small managed Postgres (RDS t4g.micro, or a free tier) per env	$30
Payments	Stripe test mode	$0
Monitoring	Grafana Cloud free tier	$0
Logs, registry, DNS	CloudWatch, ECR, Route 53	$15
Contingency (~20%)	Reruns, forgotten resources	$20
Total	No load balancer or NAT needed; the managed container service handles HTTPS	**≈ $110**

Guardrails: AWS Budget of $200/month with alerts at 50% and 80%; staging is scaled to zero between test days.

07 Team we need

What everyone does

- Writes code, reviews someone else's pull requests, deploys through dev → staging → prod
- Joins the weekly sync, logs decisions, presents their own area at the defence

Roles

Role	Owner	Leads
Platform / DevOps	Abhinav	Cloud setup, environments, CI/CD, budget, rollback
Backend	*[name]*	Webhook inbox, events table, order worker, retries and replay
Test & Reliability	*[name]*	Load generator, chaos tests, dashboard, report and slides

08 10-week plan

Week	Phase	Deliverable	Env
1–2	Problem + team	Problem statement, team, this plan	—
3	Design	Architecture, data model, naive baseline built	Dev
4–5	Core	Inbox + events table + worker; exactly-once works locally	Dev
6	Deploy	CI/CD, staging live, dashboard	Staging
7	Chaos test #1	Sale peak + crashes; first numbers vs. baseline	Staging
8	Fix	Fix what broke; dead-letter + replay	Staging
9	Prod readiness	Promote to prod, alerts, rollback drill	Prod
10	Defence	Live chaos demo + recorded backup, report, slides	Prod

09 Risks

Risk	Response
Looks too simple	The chaos demo makes it concrete: the naive version visibly loses orders and ours doesn't
Stripe test-mode rate limits	Mix real test webhooks with signed replayed events from our generator
Live demo fails	Recorded backup run + saved dashboard

10 Why this is a good Option B

It's a real problem every online store has. The core ideas (idempotency, retries, dead-letter queues, safe deploys) are standard engineering that every backend job expects. It's cheap (≈ $110), fits 3 people, and has a clear demo: **crash the system during a sale and show that no order was lost or duplicated.**